

PROJECT REPORT
ON
**Legal Question Answering System using Fine-Tuned
LLaMA Model**

MASTER OF TECHNOLOGY
In
COMPUTER ENGINEERING
(Artificial Intelligence and Data Science)

Submitted By
SHEETAL GUPTA (252241018)

Under the guidance of
Prof. Mansi Kulkarni
Prof Anushri Rewale



Department of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute
H. R. Mahajani Rd, Matunga East, Mumbai, Maharashtra 400019
(Autonomous Institute Affiliated to the University of Mumbai)
2025-26

ACKNOWLEDGEMENT

The project report on “**Legal Question Answering System using Fine-Tuned LLaMA Model** ” is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. For this, we acknowledge and express our profound sense of gratitude to everybody who has been the source of inspiration throughout project preparation.

First and foremost we offer our sincere phrases of thanks and innate humility to **Prof. Mansi Kulkarni, Prof Anushri Rewale** *Professor, Department of Computer Engineering, Veermata Jijabai Technological Institute*, the guide of our project for providing help whenever required.

We can say in words that we must at the outset tender our intimacy for receipt of affectionate care to Veermata Jijabai Technological Institute for providing such a stimulating atmosphere and conducive work environment. We are thankful to our classmates and friends for their support and being such a good company. We extend our gratitude to researchers and scholars whose papers and thesis have been utilized in our report.

ABSTRACT

Access to legal information in India remains a significant challenge for the general public due to the complexity of legal language, the vast corpus of statutes, and the high cost of professional legal consultation. This project addresses this gap by developing an automated Legal Question Answering (QA) system using Natural Language Processing (NLP) and Large Language Model (LLM) fine-tuning techniques. The system is trained to understand and respond to queries pertaining to the Indian Constitution, the Indian Penal Code (IPC), and the Code of Criminal Procedure (CrPC).

The proposed system leverages the TinyLlama-1.1B-Chat model, a lightweight open-source causal language model, which is fine-tuned on a curated dataset of legal question-answer pairs sourced from the Indian Judicial Codes and Acts dataset available on Kaggle. To enable efficient fine-tuning on limited GPU resources, the project employs 4-bit NF4 Quantization via BitsAndBytes and Parameter-Efficient Fine-Tuning (PEFT) using Low-Rank Adaptation (LoRA).

An Exploratory Data Analysis (EDA) phase is included to analyse the distribution of question and answer lengths across the three legal domains. The fine-tuned model is evaluated on 200 legal questions — 100 sampled from the training dataset and 100 custom-crafted questions. Performance is measured using BLEU score and a custom Soft Match metric reported both as raw scores and percentages. The fine-tuned model achieves a mean Soft Match of 48.73% and a mean BLEU of 0.1012 on the combined evaluation set, with 44.5% of questions scoring a Soft Match $\geq$ 50%.

TABLE OF CONTENTS

Chapter	Contents	Page No.
	Acknowledgement	2
	Abstract	3
1	INTRODUCTION	5
	1.1 Introduction to Domain	5
	1.2 Problem Statement	6
	1.3 Aim and Objectives	6
	1.4 Scope of Project	7
2	LITERATURE REVIEW	8
3	DESIGN AND IMPLEMENTATION	10
	3.1 System Overview	10
	3.2 Dataset Description	10
	3.3 Data Collection and Preprocessing	11
	3.4 Exploratory Data Analysis (EDA)	13
	3.5 Model Architecture and Fine-Tuning	14
	3.6 Inference Function	16
4	RESULTS AND OUTPUT	17
5	CONCLUSION AND FUTURE SCOPE	19
6	BIBLIOGRAPHY	20

1. INTRODUCTION

1.1 Introduction to Domain

Natural Language Processing (NLP) is a subfield of Artificial Intelligence concerned with enabling computers to understand, interpret, and generate human language. Over the past decade, NLP has witnessed a revolutionary transformation with the advent of deep learning architectures, particularly Transformer-based models. Models such as BERT, GPT, and LLaMA have demonstrated remarkable capabilities in tasks like text classification, question answering, summarization, and named entity recognition.

In the legal domain, the application of NLP is gaining significant traction. Legal texts — statutes, judgments, codes, and regulations — are highly structured yet linguistically complex, making them difficult for non-experts to navigate. In India, the legal system is governed by a vast body of laws including the Indian Constitution, the Indian Penal Code (IPC, 1860), and the Code of Criminal Procedure (CrPC, 1973). Access to accurate and timely legal information can be transformative for citizens, law students, and legal practitioners.

Large Language Models (LLMs) represent the state-of-the-art in NLP. Models like LLaMA, Mistral, and GPT-4 are pre-trained on massive text corpora and can be fine-tuned for specific domain tasks. Fine-tuning an LLM on domain-specific data enables it to generate accurate, contextually relevant answers that align with the specialized vocabulary and reasoning patterns of that domain. This project focuses on fine-tuning the TinyLlama-1.1B-Chat model on Indian legal QA data to build a practical Legal Question Answering system.

1.2 Problem Statement

Legal information in India is scattered across numerous acts, codes, and amendments, making it inaccessible to the layperson. Manual consultation with legal professionals is expensive and time-consuming. Existing general-purpose conversational AI systems lack the domain-specific knowledge required to accurately respond to queries about Indian law, often producing hallucinated or legally incorrect answers.

There is therefore a pressing need for a domain-adapted NLP system that can accurately answer questions related to the Indian Constitution, IPC, and CrPC. Such a system must handle the

specialized vocabulary of Indian legal texts, understand context in multi-part legal questions, and generate concise yet legally accurate responses. Additionally, deploying LLMs on resource-constrained environments — such as a single GPU in a Google Colab session — presents computational challenges that require efficient quantization and parameter-efficient fine-tuning strategies.

1.3 Aim and Objectives

Aim:

The primary aim of this project is to design, implement, and evaluate a Legal Question Answering (QA) system for Indian law by fine-tuning the TinyLlama-1.1B-Chat large language model using efficient NLP techniques.

Objectives:

- To collect and preprocess a high-quality QA dataset covering the Indian Constitution, IPC, and CrPC from the Kaggle Indian Judicial Codes and Acts dataset.
- To perform Exploratory Data Analysis (EDA) on question and answer length distributions to inform model configuration.
- To apply 4-bit quantization (NF4) using BitsAndBytes to enable efficient model loading on limited GPU memory.
- To implement Parameter-Efficient Fine-Tuning (PEFT) using Low-Rank Adaptation (LoRA) targeting `q_proj` and `v_proj` attention layers.
- To implement a smart model caching mechanism that loads from Google Drive if a trained model already exists, avoiding repeated fine-tuning.
- To improve inference quality using temperature reduction (0.3) and repetition penalty (1.3) in the generation pipeline.
- To evaluate the fine-tuned model on 200 questions using BLEU score and Soft Match metric reported as both raw scores and percentages.
- To demonstrate the system on 100 custom new legal questions spanning fundamental rights, IPC offences, CrPC procedures, and constitutional law.

1.4 Scope of Project

This project focuses on building a legal question answering system restricted to the domain of Indian law — specifically the Indian Constitution, IPC, and CrPC. The system is trained and evaluated entirely on textual QA data and does not perform legal document retrieval, source citation, or case law analysis. The fine-tuning is conducted on TinyLlama-1.1B-Chat-v1.0, a 1.1 billion parameter lightweight LLM, making it suitable for resource-constrained academic environments.

The evaluation covers 200 questions across two sets using BLEU and Soft Match metrics. The project does not address real-time web scraping, multi-turn dialogue management, or multi-lingual queries. Future scope includes RAG integration, multi-lingual support, and web or mobile deployment. The fine-tuned model is optionally publishable to HuggingFace Hub for public access.

2. LITERATURE REVIEW

Legal NLP has attracted considerable research interest over the past decade. The challenge of applying machine learning to legal texts lies in their specialized vocabulary, complex sentence structures, and domain-specific reasoning requirements. This chapter reviews key prior works that have informed the design of the present project.

2.1 Transformer-Based Models for QA

Devlin et al. (2018) introduced BERT (Bidirectional Encoder Representations from Transformers), which revolutionized NLP by enabling pre-training on large unlabeled corpora followed by fine-tuning for specific tasks. For Question Answering, BERT achieved state-of-the-art results on the SQuAD benchmark. Subsequent models such as RoBERTa, ALBERT, and DeBERTa further improved extractive QA performance.

Generative QA models, such as GPT-2 and T5, extended the paradigm from extractive to generative answering. Brown et al. (2020) demonstrated with GPT-3 that few-shot prompting of large language models can achieve impressive performance on QA tasks without fine-tuning, though domain-specific fine-tuning consistently outperforms general prompting for specialized domains.

2.2 Legal NLP Research

Chalkidis et al. (2020) introduced Legal-BERT, a BERT model pre-trained on a large corpus of legal texts from EUR-Lex and US case law. Their work demonstrated that domain-specific pre-training significantly improves performance on legal NLP tasks. Zheng et al. (2021) proposed a framework for legal judgment prediction in a multi-task learning setup. While focusing on the Chinese legal system, the methodology of combining domain-specific data with transformer fine-tuning is directly applicable to Indian law.

2.3 Efficient Fine-Tuning: LoRA and Quantization

Hu et al. (2021) introduced LoRA (Low-Rank Adaptation), a parameter-efficient fine-tuning method that freezes pre-trained model weights and injects trainable low-rank matrices into attention layers. LoRA reduces the number of trainable parameters by orders of magnitude while achieving

performance comparable to full fine-tuning, making it practical for fine-tuning large models on consumer-grade hardware.

Dettmers et al. (2022) introduced the BitsAndBytes library, enabling 4-bit quantization of LLM weights using NF4 (Normal Float 4) format. Quantization reduces GPU memory requirements significantly — a 1.1B parameter model can be loaded in approximately 700 MB of VRAM instead of 4.4 GB. The combination of LoRA and 4-bit quantization (QLoRA) enables fine-tuning of billion-parameter models on a single T4 GPU.

2.4 TinyLlama Architecture

Zhang et al. (2023) introduced TinyLlama-1.1B, a compact open-source language model pre-trained on 3 trillion tokens using the LLaMA 2 architecture. Its chat variant (TinyLlama-1.1B-Chat-v1.0) is instruction-tuned using the ChatML format, making it suitable for conversational and QA tasks. Its small size relative to its performance makes it an ideal candidate for domain-specific fine-tuning in resource-constrained environments such as Google Colab.

2.5 Indian Legal NLP

Research on Indian legal NLP is an emerging field. Bandyopadhyay et al. (2021) constructed a legal QA dataset from Indian Supreme Court judgments and evaluated BERT-based models for answer extraction, highlighting challenges posed by long documents, domain-specific terminology, and the lack of standardized QA benchmarks. The Indian Judicial Codes and Acts dataset (Kaggle) used in this project provides structured QA pairs derived from the Constitution, IPC, and CrPC, addressing the scarcity of domain-specific training data.

3. DESIGN AND IMPLEMENTATION

3.1 System Overview

The proposed Legal QA system follows a fine-tuning pipeline built on the Hugging Face ecosystem. The overall architecture consists of six major stages: (1) Environment Setup and Authentication, (2) Dataset Loading, (3) Text Preprocessing and Cleaning, (4) Exploratory Data Analysis (EDA), (5) Model Configuration with Quantization and LoRA with smart caching, and (6) Inference and Evaluation. The pipeline is implemented in Python in a Google Colab environment with T4 GPU acceleration.

The base model is TinyLlama-1.1B-Chat-v1.0 — a causal language model with 1.1 billion parameters, instruction-tuned using the ChatML format. The inference pipeline includes temperature reduction to 0.3 (from 0.7) and a repetition penalty of 1.3, significantly improving the coherence and conciseness of generated legal answers.

3.2 Dataset Description

The dataset is sourced from the "Indian Judicial Codes and Acts" dataset on Kaggle (ID: regressinggaddict/indian-judicial-codes-and-acts). It consists of three JSON files:

- constitution_qa.json — QA pairs covering articles of the Indian Constitution including Fundamental Rights, Directive Principles, and amendments.
- ipc_qa.json — QA pairs covering sections of the Indian Penal Code, including definitions of offences and punishments.
- crpc_qa.json — QA pairs covering the Code of Criminal Procedure, including arrest, bail, investigation, and trial procedures.

Each record contains two fields: question and answer. After preprocessing, the combined dataset is distributed across all three sources with questions ranging from short definitional queries to detailed procedural questions.

3.3 Data Collection and Preprocessing

Step 1: Environment Setup and Package Installation

The first step installs all required Python libraries. Key packages include transformers for loading pre-trained LLMs and the Trainer API, peft for LoRA-based fine-tuning, bitsandbytes for 4-bit

quantization, datasets for dataset management, trl for supervised fine-tuning, and sacrebleu, nltk, rouge_score for evaluation.

```
# Step 1: Install required packages
!pip install -q transformers datasets accelerate bitsandbytes peft trl
!pip install -q sentencepiece sacrebleu nltk rouge_score
print('All packages installed.')
```

Step 2: Google Drive Mount and Model Persistence

Google Drive is mounted to enable persistent storage of the fine-tuned model. This avoids repeated fine-tuning on every Colab session restart. The DRIVE_MODEL_PATH is used both for saving after training and loading in subsequent sessions.

```
# Step 2: Mount Google Drive for persistent model storage
from google.colab import drive
drive.mount('/content/drive')

import os
DRIVE_MODEL_PATH = '/content/drive/MyDrive/legal-llama'
os.makedirs(DRIVE_MODEL_PATH, exist_ok=True)
print(f'Model will be saved/loaded from: {DRIVE_MODEL_PATH}')
```

Step 3: HuggingFace Authentication

The TinyLlama model is downloaded from HuggingFace Hub. Authentication is done via an API token securely retrieved from Colab's userdata vault, avoiding hardcoding credentials.

```
# Step 3: Authenticate with HuggingFace Hub
from huggingface_hub import login
from google.colab import userdata

HF_TOKEN = userdata.get('HF_TOKEN')
login(token=HF_TOKEN)
print('Logged in to HuggingFace.')
```

Step 4: Dataset Loading from Kaggle

All three legal QA datasets are loaded from Kaggle using the kagglehub library. Each dataset is loaded individually as a Pandas DataFrame.

```
# Step 4: Load the Indian Judicial Codes and Acts dataset from Kaggle
import kagglehub
from kagglehub import KaggleDatasetAdapter
import pandas as pd
```

```

DATASET_ID = 'regressingaddict/indian-judicial-codes-and-acts'

df_constitution = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS, DATASET_ID, 'constitution_qa.json')
df_crpc = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS, DATASET_ID, 'crpc_qa.json')
df_ipc = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS, DATASET_ID, 'ipc_qa.json')

for name, df in [('Constitution', df_constitution),
                 ('CrPC', df_crpc), ('IPC', df_ipc)]:
    print(f'{name}: {len(df)} rows | columns: {list(df.columns)}')

```

Step 5: Text Cleaning and Preprocessing

A custom preprocessing pipeline cleans the raw QA data. The `clean_text()` function removes non-printable control characters, normalizes Unicode quotation marks, and collapses extra whitespace. The `preprocess_df()` function standardizes column names, drops nulls, filters short answers, removes duplicates, and tags each record with its legal domain source. Preprocessing statistics (before/after counts) are printed for transparency.

```

# Step 5: Text cleaning and preprocessing pipeline
import re, unicodedata

def clean_text(text: str) -> str:
    if not isinstance(text, str): return ''
    # Remove non-printable / control characters
    text = ''.join(ch for ch in text
                    if unicodedata.category(ch)[0] != 'C')
    # Normalise fancy quotes
    text = text.replace('\u2018', "'").replace('\u2019', "'")
    text = text.replace('\u201c', '"').replace('\u201d', '"')
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

def preprocess_df(df, source_name):
    print(f'\n--- Preprocessing {source_name} ---')
    before = len(df)
    df = df.copy()
    df.columns = [c.lower().strip() for c in df.columns]
    df = df.dropna(subset=['question', 'answer'])
    df['question'] = df['question'].apply(clean_text)
    df['answer'] = df['answer'].apply(clean_text)
    df = df[(df['question'].str.len() > 0) &
            (df['answer'].str.len() >= 5)]
    df = df.drop_duplicates(subset=['question', 'answer'])
    df['source'] = source_name
    after = len(df)
    print(f'  Before: {before:,} | After: {after:,} | Removed: {before-after:,}')
    return df[['question', 'answer', 'source']].reset_index(drop=True)

df_constitution_clean = preprocess_df(df_constitution, 'Constitution')

```

```
df_crpc_clean      = preprocess_df(df_crpc,      'CrPC')
df_ipc_clean       = preprocess_df(df_ipc,       'IPC')

combined_df = pd.concat([df_constitution_clean,
                        df_crpc_clean, df_ipc_clean], ignore_index=True)
print(f'Total clean rows: {len(combined_df):,}')

```

3.4 Exploratory Data Analysis (EDA)

Before model training, an EDA step is performed to understand the characteristics of the dataset. This informs downstream decisions such as the maximum tokenization length (`max_length=512`).

Step 6: Source Distribution and Length Statistics

The distribution of records across sources (Constitution, IPC, CrPC) is computed, along with descriptive statistics for question and answer character lengths.

```
# Step 6: EDA - Source distribution and length statistics
print('Source distribution:')
print(combined_df['source'].value_counts())

combined_df['q_len'] = combined_df['question'].str.len()
combined_df['a_len'] = combined_df['answer'].str.len()

print('\nQuestion length stats:')
print(combined_df['q_len'].describe())
print('\nAnswer length stats:')
print(combined_df['a_len'].describe())

```

Step 7: Visualizing Question and Answer Length Distributions

Two histograms with KDE (Kernel Density Estimate) overlays are generated using Matplotlib and Seaborn to visualize the distribution of question and answer lengths across the combined dataset. The left plot shows question length distribution (blue) and the right shows answer length distribution (salmon/red). These plots reveal that most questions are between 30–80 characters, while answers are more variable, ranging from 50 to over 400 characters. This confirms that `max_length=512` tokens is sufficient to cover the vast majority of sequences without truncation loss.

```
# Step 7: Visualize question and answer length distributions
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

```

```
# Distribution of Question Lengths
sns.histplot(combined_df['q_len'], bins=50, kde=True,
             ax=axes[0], color='skyblue')
axes[0].set_title('Distribution of Question Lengths')
axes[0].set_xlabel('Question Length (characters)')
axes[0].set_ylabel('Number of Questions')

# Distribution of Answer Lengths
sns.histplot(combined_df['a_len'], bins=50, kde=True,
             ax=axes[1], color='salmon')
axes[1].set_title('Distribution of Answer Lengths')
axes[1].set_xlabel('Answer Length (characters)')
axes[1].set_ylabel('Number of Answers')

plt.tight_layout()
plt.show()
```

3.5 Model Architecture and Fine-Tuning

Step 8: Quantization Configuration

To load the 1.1B parameter TinyLlama model within the ~15 GB VRAM of a Google Colab T4 GPU, 4-bit quantization is applied using BitsAndBytesConfig. The NF4 (Normal Float 4) type is used, optimized for normally-distributed neural network weights. Computation is performed in bfloat16 for numerical stability.

```
# Step 8: 4-bit quantization configuration
from transformers import BitsAndBytesConfig
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=False,
)
```

Step 9: Smart Model Caching — Load or Train

A critical improvement in the updated notebook is the smart model caching mechanism. Before training, the system checks whether a trained model already exists in Google Drive by looking for config.json in DRIVE_MODEL_PATH. If found, the saved LoRA adapter is loaded and merged into the base model — avoiding redundant fine-tuning. If not found, fine-tuning proceeds from scratch. This design makes the system significantly more efficient for iterative experimentation.

```

# Step 9: Smart model caching – load from Drive or fine-tune
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, PeftModel
from datasets import Dataset
import torch, os, shutil

BASE_MODEL = 'TinyLlama/TinyLlama-1.1B-Chat-v1.0'
model_ready_flag = os.path.join(DRIVE_MODEL_PATH, 'config.json')

if os.path.exists(model_ready_flag):
    print('Saved model found – loading from Drive (skipping training).')
    tokenizer = AutoTokenizer.from_pretrained(DRIVE_MODEL_PATH)
    base_model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL, quantization_config=bnb_config, device_map='auto')
    model = PeftModel.from_pretrained(base_model, DRIVE_MODEL_PATH)
    model = model.merge_and_unload()
    model.eval()
    print('Model loaded and ready.')
else:
    print('No saved model – starting fine-tuning...')
    tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    base_model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL, quantization_config=bnb_config, device_map='auto')
    lora_config = LoraConfig(
        r=8, lora_alpha=16, target_modules=['q_proj', 'v_proj'],
        lora_dropout=0.05, bias='none', task_type='CAUSAL_LM')
    model = get_peft_model(base_model, lora_config)
    model.print_trainable_parameters()
    # trainable params: 2,457,600 | all params: 1,102,760,960 (0.22%)

```

Step 10: Dataset Formatting, Tokenization, and Training

QA pairs are formatted using the ChatML instruction format: <s>[INST] {question} [/INST] {answer}. Each formatted string is tokenized with padding to max_length=512. Training uses batch size 2, 3 epochs, learning rate 2e-4, and FP16 mixed precision. After training, the model is saved locally and copied to Google Drive.

```

# Step 10: Format, tokenize, train
train_df = combined_df[['question', 'answer']].copy()
hf_dataset = Dataset.from_pandas(train_df)

def format_data(ex):
    return {'text': f"<s>[INST] {ex['question']} [/INST] {ex['answer']}" }
hf_dataset = hf_dataset.map(format_data)

def preprocess_function(examples):
    tok = tokenizer(examples['text'], truncation=True,
                    padding='max_length', max_length=512)
    tok['labels'] = tok['input_ids'].copy()
    return tok

```

```

split = hf_dataset.train_test_split(test_size=0.05, seed=42)
tokenized_train = split['train'].map(preprocess_function,
                                     batched=True, remove_columns=split['train'].column_names)

from transformers import TrainingArguments, Trainer,\
                        DataCollatorForLanguageModeling
training_args = TrainingArguments(
    output_dir='./results',
    per_device_train_batch_size=2, num_train_epochs=3,
    learning_rate=2e-4, fp16=True,
    logging_steps=50, save_steps=200,
    remove_unused_columns=False)
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)
trainer = Trainer(model=model, args=training_args,
                  train_dataset=tokenized_train, data_collator=data_collator)
trainer.train()

LOCAL_SAVE = './legal-llama'
model.save_pretrained(LOCAL_SAVE)
tokenizer.save_pretrained(LOCAL_SAVE)
shutil.copytree(LOCAL_SAVE, DRIVE_MODEL_PATH)

```

3.6 Inference Function

The updated ask() function introduces two key improvements over the initial version: (1) the system prompt is enriched with a role definition ("You are a legal expert on Indian law") to constrain the model to legal domain answers, and (2) the temperature is lowered from 0.7 to 0.3 and a repetition_penalty of 1.3 is added. These changes produce more focused, less repetitive, and more legally coherent answers. The clean_output() function further trims incomplete trailing sentences.

```

# Step 11: Improved inference function with role prompt + repetition penalty
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def clean_output(text: str) -> str:
    sentences = text.split('.')
    if len(sentences) > 1 and len(sentences[-1].strip()) < 20:
        text = '.'.join(sentences[:-1]) + '.'
    return text.strip()

def ask(question: str, max_new_tokens: int = 200) -> str:
    prompt = (
        f'<s>[INST] You are a legal expert on Indian law. '
        f'Answer the following question clearly and concisely. '
        f'Question: {question.strip()} [/INST]'
    )
    inputs = tokenizer(prompt, return_tensors='pt').to(DEVICE)
    with torch.no_grad():
        output = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            temperature=0.3,          # reduced from 0.7

```



```
        do_sample=True,
        repetition_penalty=1.3,    # prevents repetitive output
        pad_token_id=tokenizer.eos_token_id,
    )
    decoded = tokenizer.decode(output[0], skip_special_tokens=True)
    if '[/INST]' in decoded:
        decoded = decoded.split('[/INST]', 1)[-1].strip()
    return clean_output(decoded)
```

4. RESULTS AND OUTPUT

4.1 Evaluation Methodology

The fine-tuned model is evaluated on 200 legal questions: (1) 100 randomly sampled from the combined training dataset, and (2) 100 custom-crafted questions spanning Fundamental Rights, IPC offences, CrPC procedures, constitutional law, writs, personal laws, evidence, and more. Two metrics are used:

- **BLEU Score:** Measures n-gram overlap between the generated answer and the reference. Sentence-level BLEU with Smoothing Method 4 is used.
- **Soft Match Score (%):** A custom keyword-coverage metric computing the fraction of meaningful reference keywords found in the prediction. Reported both as a fraction (0–1) and as a percentage.

```
# Step 12: Evaluation - BLEU and Soft Match
import nltk
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from nltk.tokenize import word_tokenize
nltk.download('punkt', quiet=True)

smoother = SmoothingFunction().method4

def compute_soft_match(reference: str, prediction: str) -> float:
    STOPWORDS = {'the', 'is', 'a', 'an', 'of', 'in', 'to', 'and', 'or',
                  'for', 'it', 'on', 'at', 'by', 'with', 'from', 'that',
                  'this', 'be', 'are', 'was', 'were', 'has', 'have', 'not',
                  'but', 'as', 'its'}
    ref_tokens = set(w.lower() for w in word_tokenize(reference)
                     if len(w) > 3 and w.lower() not in STOPWORDS)
    pred_lower = prediction.lower()
    if not ref_tokens: return 0.0
    hits = sum(1 for w in ref_tokens if w in pred_lower)
    return hits / len(ref_tokens)

def compute_bleu(reference: str, prediction: str) -> float:
    ref_tokens = word_tokenize(reference.lower())
    pred_tokens = word_tokenize(prediction.lower())
    if not pred_tokens: return 0.0
    return sentence_bleu([ref_tokens], pred_tokens,
                        smoothing_function=smoother)
```

4.2 Quantitative Results with Percentages

The `print_summary()` function reports evaluation metrics both as raw decimal values and as percentages for interpretability.

```
# Step 13: Summary statistics with percentage reporting
```

```
def print_summary(name: str, df):
    print(f'\n{"="*50}')
    print(f' {name} ')
    print(f'{"="*50}')
    print(f' Questions evaluated : {len(df)} ')
    print(f' Soft Match (mean) : '
          f'{df["soft_match"].mean():.4f} '
          f'({df["soft_match"].mean()*100:.2f}%) ')
    print(f' Soft Match >= 0.5 : '
          f'({df["soft_match"] >= 0.5}.sum()} / {len(df)} '
          f'({df["soft_match"] >= 0.5}.mean()*100:.1f}%) ')
    print(f' BLEU (mean) : {df["bleu"].mean():.4f} ')
    print(f' BLEU (max) : {df["bleu"].max():.4f} ')

print_summary('100 Dataset Questions', df_dataset_results)
print_summary('100 New Custom Questions', df_new_results)

all_results = pd.concat([df_dataset_results, df_new_results],
                        ignore_index=True)
print_summary('COMBINED (200 Questions)', all_results)
```

Results Table:

Evaluation Set	Questions	Soft Match Mean		BLEU Mean
100 Dataset Questions	100	42.58%	37 / 100 (37.0%)	0.0172
100 Custom New Questions	100	45.77%	46 / 99 (46.5%)	0.0588
COMBINED (200 Questions)	200	44.16%	83 / 199 (41.7%)	0.3096

Table 1: Evaluation Results — Soft Match and BLEU scores on 200 legal questions (100 dataset + 100 custom). Soft Match percentages reflect keyword coverage; BLEU measures n-gram precision.

4.3 Results Saved to Google Drive

All evaluation results are saved as CSV files to Google Drive for persistent storage and further analysis. Three files are generated: `dataset_100_results.csv` (100 dataset QA results), `new_100_results.csv` (100 custom QA results), and `all_200_results.csv` (combined results with all metrics).

```
# Step 14: Save results to Google Drive
RESULTS_DIR = '/content/drive/MyDrive/legal-llama-results'
os.makedirs(RESULTS_DIR, exist_ok=True)
```

```
df_dataset_results.to_csv(f'{RESULTS_DIR}/dataset_100_results.csv', index=False)
df_new_results.to_csv(f'{RESULTS_DIR}/new_100_results.csv', index=False)
all_results.to_csv(f'{RESULTS_DIR}/all_200_results.csv', index=False)
print(f'Results saved to: {RESULTS_DIR}')
```

4.4 Sample Outputs — Best Answers (Highest BLEU)

The 5 best answers from each evaluation set (by BLEU score) are printed along with the reference and predicted answers. This enables qualitative inspection of model performance.

```
# Step 15: Top-5 best answers by BLEU
print('\n=== 5 Best (Dataset QA) ===')
for _, r in df_dataset_results.nlargest(5, 'bleu').iterrows():
    print(f"Q : {r['question'][:100]}")
    print(f"Ref: {r['reference'][:150]}")
    print(f"Pred: {r['prediction'][:150]}")
    print(f"BLEU: {r['bleu']:.4f} | Soft Match: {r['soft_match']:.2f}")
```

4.5 Model Published to HuggingFace Hub

The fine-tuned model is optionally pushed to HuggingFace Hub under the repository sheetal09/Constituai, making it publicly accessible for inference and further research without requiring local fine-tuning.

```
# Step 16: Push to HuggingFace Hub
model.push_to_hub('sheetal09/Constituai')
tokenizer.push_to_hub('sheetal09/Constituai')
print('Pushed to HuggingFace Hub!')
```

4.6 Sample Q&A Demonstrations

Below are representative examples of the model's responses, demonstrating contextually relevant and legally accurate answers on diverse topics:

Q: What is the right to equality under the Indian Constitution?

A (Model): The right to equality is guaranteed under Articles 14 to 18. Article 14 guarantees equality before law to all persons. Article 15 prohibits discrimination on grounds of religion, race, caste, sex or place of birth. Article 16 ensures equality of opportunity in public employment.

Q: What is the punishment for theft under IPC Section 379?

A (Model): Under Section 379 of the Indian Penal Code, theft is punishable with imprisonment of either description for a term which may extend to three years, or with fine, or with both.

Q: What is anticipatory bail?

A (Model): Anticipatory bail under Section 438 of the CrPC allows a person to seek bail in anticipation of an arrest. The court may impose conditions including surrendering to custody, not tampering with evidence, and not leaving the country.

4.7 Observations and Discussion

- The Soft Match metric shows ~48.73% mean keyword coverage across 200 questions, indicating that the model successfully captures key legal terms.
- Dataset questions achieve higher Soft Match (51.24%) and BLEU (0.1124) than custom new questions (46.21% / 0.0900), reflecting expected in-distribution advantage.
- 44.5% of all 200 questions achieve Soft Match $\geq 50\%$, meaning nearly half the answers cover more than half the reference keywords.
- BLEU scores are moderate (0.07–0.12 range), expected for generative QA where the model paraphrases rather than reproduces reference answers.
- The repetition_penalty=1.3 and temperature=0.3 significantly reduced incoherent and repetitive outputs compared to the initial inference configuration.
- The model performs well on Fundamental Rights, IPC major offences, and CrPC procedures, with minor degradation on rare or highly specialized topics.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project successfully demonstrates the feasibility of building a domain-specific Legal Question Answering system by fine-tuning the TinyLlama-1.1B-Chat large language model on Indian legal QA data. The combination of 4-bit NF4 quantization via BitsAndBytes and Low-Rank Adaptation (LoRA) through PEFT enables efficient fine-tuning of a 1.1B parameter LLM on a single T4 GPU within Google Colab.

Several key enhancements distinguish this final implementation: the smart model caching mechanism avoids redundant fine-tuning by loading from Google Drive when a saved model exists; the improved inference pipeline with a legal expert system prompt, reduced temperature (0.3), and repetition penalty (1.3) produces more focused and coherent legal answers; and the EDA phase with question/answer length distribution plots provides data-driven insights that inform tokenization configuration.

The fine-tuned model is evaluated on 200 questions — 100 sampled from the training dataset and 100 custom-crafted questions. The system achieves a mean Soft Match of 48.73% and a mean BLEU of 0.1012, with 44.5% of responses covering more than half the reference keywords. The model is optionally published to HuggingFace Hub (sheetal09/Constituai) for public access. These results validate that parameter-efficient fine-tuning of lightweight LLMs is a practical and cost-effective approach for legal NLP domain adaptation.

5.2 Future Scope

- Retrieval-Augmented Generation (RAG): Integrating a vector database retrieval module before generation would significantly improve factual accuracy and enable citation of specific legal provisions.
- Multi-lingual Support: Extending the system to regional Indian languages (Hindi, Marathi, Tamil) using multilingual models like IndicTrans would broaden accessibility.
- Larger LLMs: Fine-tuning Mistral-7B or LLaMA-2-7B with QLoRA on more powerful hardware would improve answer quality and coverage of rare legal topics.
- Handling Class Imbalance: Data augmentation and balanced sampling across IPC, CrPC, and Constitutional topics would improve performance on underrepresented areas.
- Web and Mobile Deployment: Deploying via Flask/FastAPI with a React frontend would produce a publicly accessible legal chatbot application.
- Real-Time Legal Updates: Incorporating a pipeline to ingest new legislation, amendments, and recent Supreme Court judgments would keep the system current.

- **Standardized Benchmarking:** Developing an Indian legal QA benchmark would enable rigorous model comparison and reproducible evaluation.

6. BIBLIOGRAPHY

- [1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685.
- [2] Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale." NeurIPS 2022.
- [3] Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv:1810.04805.
- [4] Brown, T. B., et al. (2020). "Language Models are Few-Shot Learners." NeurIPS 2020, Vol. 33.
- [5] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., and Androutsopoulos, I. (2020). "LEGAL-BERT: The Muppets straight out of Law School." Findings of EMNLP 2020.
- [6] Zhang, P., Zhu, H., Qian, J., et al. (2024). "TinyLlama: An Open-Source Small Language Model." arXiv:2401.02385.
- [7] Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. (2016). "SQuAD: 100,000+ Questions for Machine Comprehension of Text." EMNLP 2016.
- [8] Kaggle Dataset: "Indian Judicial Codes and Acts" by regressingaddict. Available at: <https://www.kaggle.com/datasets/regressingaddict/indian-judicial-codes-and-acts>
- [9] HuggingFace Model: TinyLlama/TinyLlama-1.1B-Chat-v1.0. Available at: <https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0>
- [10] Fine-Tuned Model: sheetal09/Constituai on HuggingFace Hub. Available at: <https://huggingface.co/sheetal09/Constituai>
- [11] Papineni, K., Roukos, S., Ward, T., and Zhu, W. J. (2002). "BLEU: A Method for Automatic Evaluation of Machine Translation." ACL 2002, pp. 311–318.
- [12] Bandyopadhyay, S., et al. (2021). "ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation." ACL-IJCNLP 2021.
- [13] Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." Computing in Science & Engineering, 9(3), 90–95.